

Takealot ERP 数据安全指南

版本：V0.1 · 适用环境：Docker 全栈部署（MySQL + NestJS 后端 + Vue 前端）

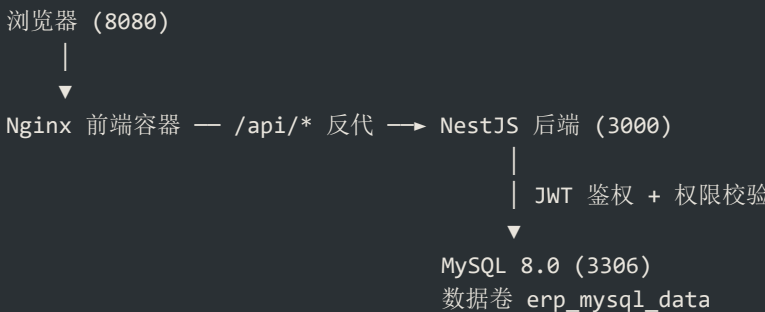
最后更新：2026-06-26

1. 文档目的

本文档说明 **Takealot ERP 当前版本** 已具备的数据安全机制、运维人员必须遵守的操作规范，以及上线生产环境前的加固清单。目标是：

- 防止未授权用户访问或篡改业务数据
- 保证账号、权限与业务数据的一致性
- 在故障、误操作或设备更换时能够恢复数据
- 明确当前系统的安全边界与已知风险

2. 系统架构与安全边界



组件	默认端口	数据存放位置
前端	8080	静态资源（无业务数据）
后端	3001 → 3000	无持久化，仅连接数据库
MySQL	3307 → 3306	Docker 卷 <code>erp_mysql_data</code>

安全边界原则：

- 所有业务数据仅存储于 MySQL，不写入浏览器 localStorage（除登录 Token 与少量 UI 偏好）
- 除登录接口外，所有 `/api/*` 请求必须携带有效 JWT
- 写操作（创建、修改、删除、审核、推送等）在后端强制校验权限码

3. 已实施的安全机制

3.1 身份认证 (Authentication)

机制	说明
JWT 登录	<code>POST /api/auth/login</code> ，校验用户名 + bcrypt 密码哈希
Token 存储	前端 <code>localStorage.token</code> ，请求头 <code>Authorization: Bearer</code>
全局守卫	<code>JwtAuthGuard</code> 默认拦截全部接口，仅 <code>@Public()</code> 标记的接口可匿名访问
会话失效	Token 过期或 401 时前端自动清除 Token 并跳转登录页
禁用账号	<code>sys_user.status = 0</code> 的账号无法登录，返回「账号已被禁用」

当前唯一公开接口：`POST /api/auth/login`

密码策略：

- 数据库存储 bcrypt 哈希（cost=10），明文密码不落库
- 新建/重置密码最少 6 位（后端 DTO 校验）
- 用户可在「个人设置」修改密码，需验证当前密码

3.2 权限控制 (Authorization)

系统采用 **角色模板 + 用户级覆盖** 的 RBAC 模型：



层级	实现
后端强制校验	全局 <code>PermissionsGuard</code> + <code>@RequirePerms('xxx.yyy')</code> 装饰器
权限数据来源	MySQL 表 <code>sys_role_permission</code> 、 <code>sys_user_permission</code>
管理员特权	<code>roleCode = admin</code> 的用户在后端守卫层自动放行
前端 UI 控制	菜单与按钮按登录用户 <code>permissions[]</code> 显示/隐藏（辅助，不可代替后端）
403 处理	前端 API 客户端捕获 403 并提示「无权限执行此操作」

权限粒度示例（共 56 项）：

- 查看类：`products.view`、`purchase.view`、`permissions.view` ...
- 操作类：`purchase.po_audit`、`product_audit.approve`、`billing.generate` ...
- 管理类：`permissions.manage`（账号开通、编辑、禁用、权限分配）

关键业务隔离示例：

- 采购员仅能看到分配给自己的「待采购 SKU」（后端 `purchase.service` 按登录用户过滤）
- 无 `permissions.manage` 权限的用户无法修改他人账号或权限
- 财务审核、主管审核、产品审核等写操作均有独立权限码

3.3 账号生命周期管理

操作	入口	权限要求	安全约束
开通账号	权限管理 → 开通账户	<code>permissions.manage</code>	可设初始状态为「禁用」
编辑资料/权限	权限管理 → 编辑	<code>permissions.manage</code>	修改后立即写入数据库
禁用/启用	权限管理 → 操作列	<code>permissions.manage</code>	不能禁用当前登录账号
个人资料	个人设置	登录即可	仅能修改自己的姓名、手机、邮箱
修改密码	个人设置	登录即可	需验证当前密码

3.4 前端访问控制

机制	说明
路由守卫	未登录用户访问任意业务页 → 重定向 <code>/login</code>
菜单过滤	侧栏按 <code>hasPerm()</code> / <code>canViewScreen()</code> 动态渲染
权限来源	登录后从 <code>GET /api/auth/profile</code> 获取， 不使用 本地演示账号权限
Token 失效	401 自动登出，避免过期 Token 继续操作

3.5 API 与输入安全

机制	说明
全局校验	<code>ValidationPipe</code> 白名单过滤 + 类型转换
统一响应	业务异常通过 <code>HttpExceptionFilter</code> 标准化，不暴露堆栈
ORM 访问	Prisma 参数化查询，降低 SQL 注入风险
CORS	当前为 <code>origin: true</code> （开发友好），生产需收紧

4. 数据存储与备份

4.1 数据持久化

所有 ERP 业务数据保存在 Docker 命名卷 `erp_mysql_data` 中，包括：

- 用户、角色、权限
- 商品、供应商、采购单、入库单
- 线索、客户、结算、库存、同步日志等

注意： 执行 `docker compose down` **不会**删除数据卷；但 `docker volume rm erp_mysql_data` 或 `docker compose down -v` **会永久删除数据**。

4.2 日常备份（推荐每日执行）

方式一：逻辑备份（推荐，体积小、可跨版本恢复）

```
# 在项目根目录执行，生成带日期的 SQL 文件
$date = Get-Date -Format "yyyyMMdd_HHmm"
docker exec erp-mysql mysqldump -uroot -perp_root_pass --single-transaction --routines takealot_erp > ".\backups\takealot_erp_$date.sql"
```

方式二：数据卷快照（完整二进制备份）

```
$date = Get-Date -Format "yyyyMMdd"
docker run --rm `
  -v erp_erp_mysql_data:/data `
  -v ${PWD}/backups:/backup `
  alpine tar czf "/backup/mysql_volume_$date.tar.gz" -C /data .
```

请先创建 backups 目录： `mkdir backups`

4.3 数据恢复

从 SQL 文件恢复：

```
# 恢复前建议先备份当前数据
docker exec -i erp-mysql mysql -uroot -perp_root_pass takealot_erp < ".\backups\takealot_erp_20260626.sql"
```

从数据卷快照恢复（需停止 MySQL）：

```
docker compose stop mysql
docker run --rm -v erp_erp_mysql_data:/data -v ${PWD}/backups:/backup alpine sh -c "rm -rf /data/* && tar xzf /backup/mysql_volume_20260626.tar.gz -C /data"
docker compose start mysql
```

4.4 备份保留策略（建议）

类型	频率	保留时长
全量 SQL 备份	每日	30 天
变更前手动备份	升级/seed/大批量导入前	永久保留最近 5 份
异地拷贝	每周	拷贝至另一台机器或云存储

4.5 禁止在生产环境随意执行的操作

操作	风险
<code>docker compose --profile seed run --rm seed</code>	会重置演示用户密码为 <code>123456</code> , 覆盖权限模板
<code>docker compose down -v</code>	删除全部数据库
直接修改 MySQL 表结构而不备份	可能导致数据不一致或丢失
将 3307/3001/8080 暴露到公网且无防火墙	数据库与 API 可被扫描攻击

5. 部署环境安全

5.1 生产环境必须修改的默认配置

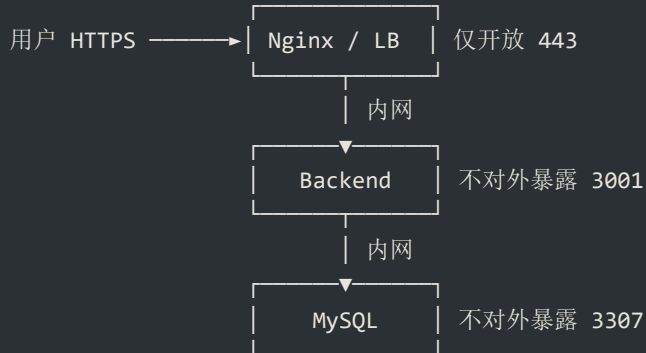
当前 `docker-compose.yml` 含 **演示用默认值**，上线前 **必须** 修改：

配置项	当前默认值	要求
<code>MYSQL_ROOT_PASSWORD</code>	<code>erp_root_pass</code>	改为 ≥ 16 位强密码
<code>JWT_SECRET</code>	<code>change-me-in-production</code>	改为随机 32+ 字符字符串
演示账号密码	seed 默认 <code>123456</code>	全部改为独立强密码
MySQL 端口映射	<code>3307:3306</code>	生产建议 不映射 到宿主机，仅容器内网访问
后端端口	<code>3001:3000</code>	生产建议仅通过 Nginx/反向代理暴露 443

生成随机 JWT Secret 示例 (PowerShell)：

```
[Convert]::ToBase64String((1..32 | ForEach-Object { Get-Random -Maximum 256 })))
```

5.2 网络安全建议



- 启用 HTTPS (Let's Encrypt 或企业证书)
- 配置防火墙，仅允许运维 IP 访问 SSH
- 定期更新 Docker 镜像与系统补丁

5.3 密钥与凭证管理

- **禁止** 将 `.env`、数据库密码、JWT Secret 提交到 Git
- 使用环境变量或 Docker Secrets 注入敏感配置
- 运维人员离职后立即：禁用账号 → 轮换 JWT Secret (使旧 Token 全部失效) → 修改数据库密码

6. 日常运维检查清单

每日

- [] 确认 `docker compose ps` 三个服务均为 `running`
- [] 确认备份任务成功 (检查 `backups/` 目录最新文件)
- [] 查看是否有异常 403/401 日志激增 (可能表示越权尝试)

每周

- [] 审查权限管理中的账号列表，禁用离职人员账号
- [] 确认无多余 `admin` 级账号
- [] 将备份复制到异地存储
- [] 检查磁盘空间 (MySQL 卷与备份目录)

每次发版前

- [] 执行全量 SQL 备份

- [] 在测试环境验证迁移/seed 脚本
- [] 确认本次变更不涉及覆盖生产权限数据
- [] 发版后抽查：普通角色登录 → 确认菜单与 API 权限符合预期

每次发版命令（安全做法）

```
# 1. 备份
docker exec erp-mysql mysqldump -uroot -perp_root_pass --single-transaction takealot_erp > .\backups\pre_deploy.sql

# 2. 重建应用（不动数据卷）
docker compose up -d --build backend frontend

# 3. 切勿随意执行 seed（除非明确要重置演示数据）
```

7. 权限管理操作规范

7.1 最小权限原则

- 新用户默认按 **岗位角色模板** 分配权限，避免直接给「系统管理员」
- 仅在确有跨模块需求时，通过「编辑 → 勾选权限」做个性化调整
- 个性化权限写入 `sys_user_permission` 后，**优先于**角色模板生效

7.2 角色与岗位对应（当前系统）

角色代码	岗位	典型权限范围
admin	系统管理员	全部
ops_manager	采购/运营主管	审核、分配、采购审批
purchaser	采购员	采购下单、收货相关
finance	财务	财务审核、结算、报表
cs	销售	线索与成交
warehouse	仓库	入库、库存
viewer / dev	产品开发	选品与开发
coach	陪跑	定价与 OMS

7.3 账号禁用流程

- 1. 权限管理 → 找到目标账号 → 点击 禁用
- 2. 确认该用户无进行中关键业务（如待审 PO）
- 3. 禁用后用户立即无法登录；已签发 JWT 在过期前仍有效（默认 7 天）→ 重要账号禁用后建议轮换 JWT Secret 或等待 Token 过期

8. 已知风险与改进建议

8.1 当前已知风险

风险	严重度	说明
默认弱密码	高	seed 账号统一为 123456， Docker MySQL root 密码为默认值
JWT Secret 默认值	高	可被伪造 Token
MySQL 端口对外映射	中	3307 暴露在本机所有网卡
CORS 全开放	中	origin: true 允许任意来源
无操作审计日志	中	谁何时改了哪条数据，暂无独立审计表
无登录失败锁定	低	可暴力尝试密码
Dashboard 等部分接口无细粒度权限	低	已登录即可访问工作台统计
Token 存 localStorage	低	存在 XSS 窃取 Token 风险（需保证前端无 XSS 漏洞）

8.2 建议后续增强（按优先级）

1. P0 — 上线前必做

- 修改所有默认密码与 JWT Secret
- 关闭 MySQL/Backend 对公网端口映射
- 启用 HTTPS

1. P1 — 短期

- 增加操作审计日志（用户 ID、接口、关键字段变更、时间）
- 登录失败 N 次锁定账号 15 分钟
- CORS 限制为前端域名白名单

1. P2 — 中期

- Token 改 HttpOnly Cookie + CSRF 防护
- 数据库只读账号供报表查询
- 自动备份 + 备份恢复演练（每季度一次）

9. 安全事件响应

9.1 疑似账号泄露

1. 立即在权限管理中 **禁用** 该账号
2. 修改 `JWT_SECRET` 并重启 backend（使所有 Token 失效）
3. 强制相关人员重新登录并修改密码
4. 检查近期异常 API 调用与数据变更

9.2 疑似数据库损坏或误删

1. **停止写入：** `docker compose stop backend`
2. 从最近备份恢复（见 4.3 节）
3. 验证核心表记录数与用户权限
4. 恢复 backend 服务

9.3 服务器硬盘故障 / 换机迁移

1. 若有最新 SQL 备份：在新机器部署 Docker Compose → 恢复 SQL
2. 若有数据卷 tar 包：挂载新卷后解压恢复
3. 重新配置环境变量（密码、JWT Secret 保持一致或统一轮换）
4. 验证登录与各模块读写

10. 快速参考

服务地址（本地 Docker 默认）

服务	地址
前端	http://localhost:8080
后端 API	http://localhost:3001/api
MySQL	localhost:3307 (user: root)

关键文件路径

文件	用途
docker-compose.yml	服务编排与环境变量
backend/prisma/schema.prisma	数据库模型
backend/src/common/permissions/permissions.constants.ts	角色权限模板
backend/src/common/guards/permissions.guard.ts	后端权限守卫
frontend/src/stores/app.ts	前端权限与登录状态
frontend/src/pages/PermissionsView.vue	账号与权限管理 UI

核心数据表

表名	内容
sys_user	用户账号、密码哈希、状态
sys_role / sys_permission	角色与权限定义
sys_role_permission	角色默认权限
sys_user_permission	用户个性化权限覆盖
业务表	product、purchase_order、inbound、lead、inventory 等

11. 结论

当前 Takealot ERP 已具备：

- 登录鉴权 + JWT 全接口保护
- 后端强制 RBAC 细粒度权限校验
- 账号禁用与个人密码修改
- 权限数据持久化在 MySQL（非前端 localStorage）

数据安全的关键在于运维纪律：定期备份、禁用离职账号、不在生产环境执行 seed、上线前替换全部默认密钥。按本文档第 5、6 节执行，可显著降低当前系统的数据丢失与未授权访问风险。

如有安全相关问题或需补充审计日志、HTTPS 部署方案，请联系系统管理员或开发团队。